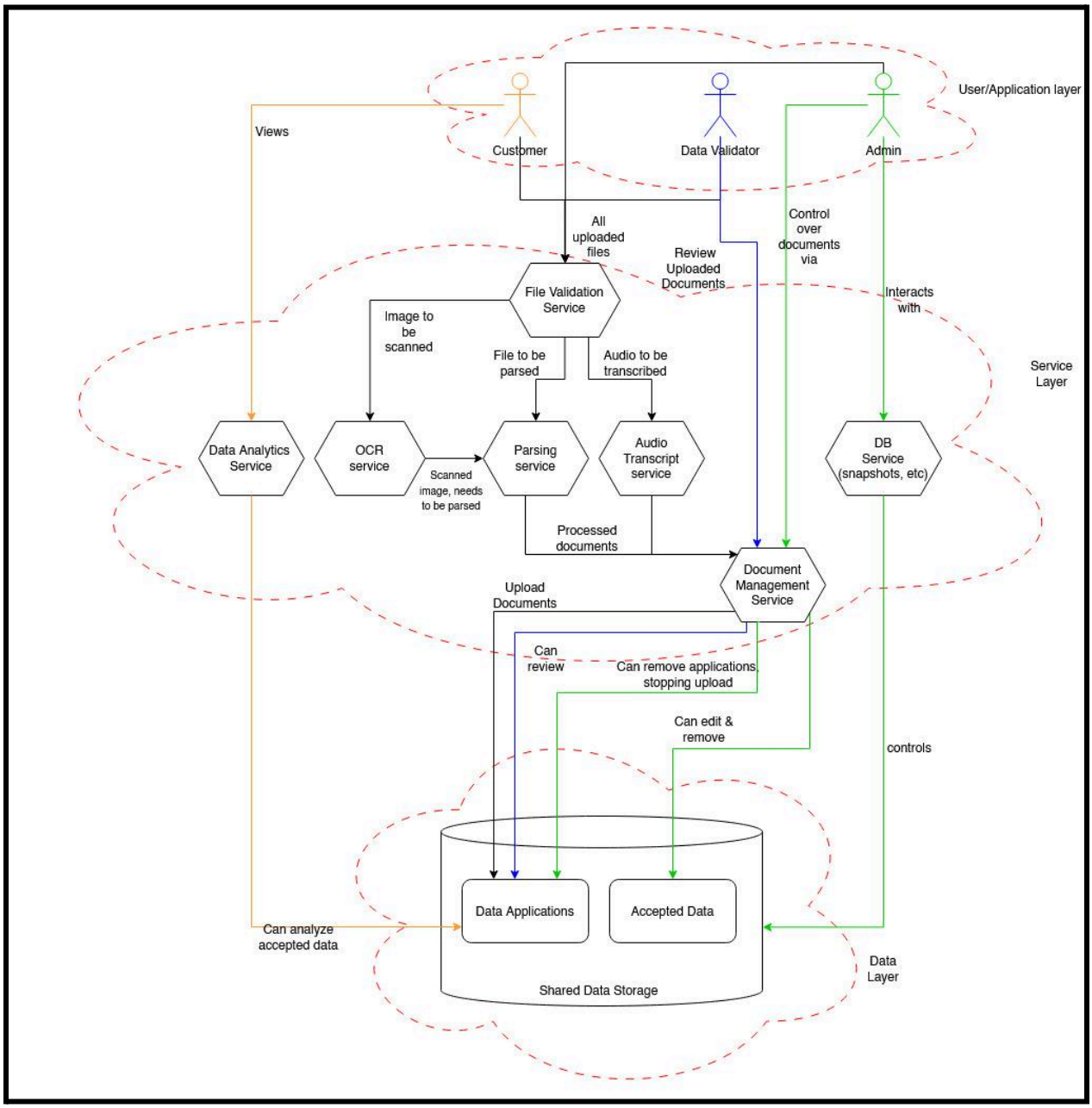
System-Level Architecture Overview



Cloud components can significantly improve the existing OCR and parsing system, which currently lacks scalability, automation, workflow tracking, and analytics. To address these issues, Amazon S3 is used to store raw files such as CSVs, images, and audio, providing a scalable and reliable storage solution. Amazon SQS is introduced to handle incoming data by buffering requests and decoupling file uploads from processing, allowing the system to manage high loads more efficiently. AWS Lambda enables event-driven automation by triggering OCR, parsing, and validation processes when files are uploaded, removing the need for manual intervention and improving scalability.

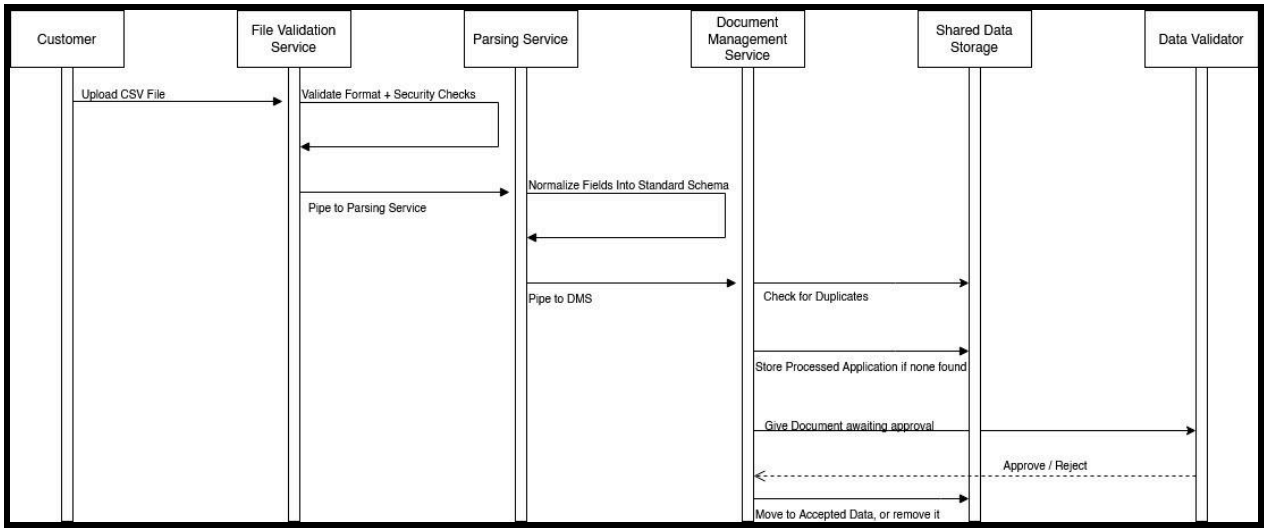
To manage the multi-step pipeline (OCR → Parsing → Cleaning → Validation → Storage), AWS Step Functions are used to orchestrate workflows, track progress, and handle failures between steps. For audio inputs, Amazon Transcribe converts speech into text automatically, improving processing efficiency. Processed data is stored in Amazon RDS or DynamoDB, which provide scalable and structured data storage. Amazon API Gateway and AWS Amplify are used to connect and support the frontend interface, ensuring smooth communication between users and backend services. Finally, Amazon QuickSight is included to provide analytics such as submission trends and error rates, which were not available in the original system.

Overall, these components transform the system into a scalable, automated, and event-driven architecture. The use of cloud services also follows an OpEx cost model, where costs scale with usage, making it more flexible and cost-effective compared to traditional upfront infrastructure (CapEx).

High-Level Sequence Diagrams (Scenario Walkthroughs)

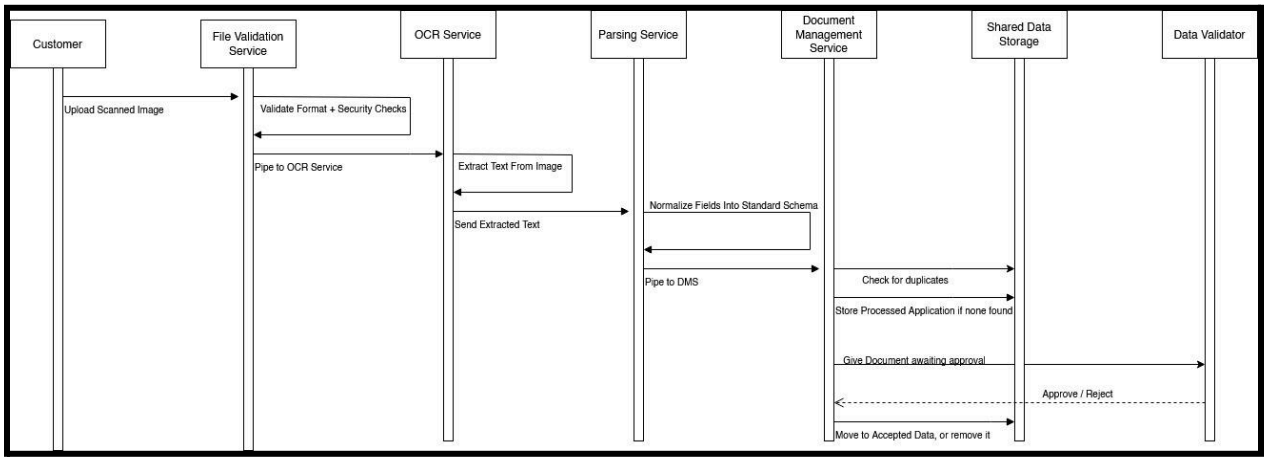
Scenario 1: CSV File Ingestion and Processing

- A Customer uploads a CSV file to the File Validation Service, which validates the format and runs security checks. The file is then piped to the Parsing Service to normalize fields into a standard schema, then to the DMS to check for duplicates. If none are found, the application is stored and forwarded to a Data Validator for an Approve/Reject decision, after which it is moved to Accepted Data or removed.



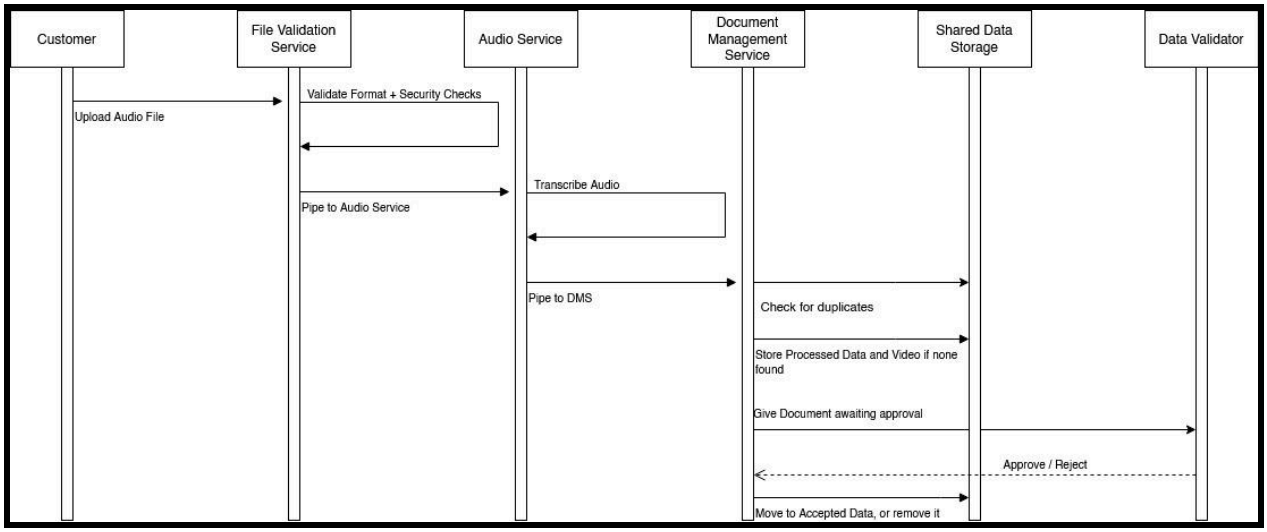
Scenario 2: Scanned Document OCR Processing

- A Customer uploads a scanned image to the File Validation Service, which validates the format and runs security checks. The file is piped to the OCR Service to extract text, which is then sent to the Parsing Service for normalization. The parsed data is forwarded to the DMS to check for duplicates. If none are found, the application is stored and forwarded to a Data Validator for an Approve/Reject decision, after which it is moved to Accepted Data or removed.



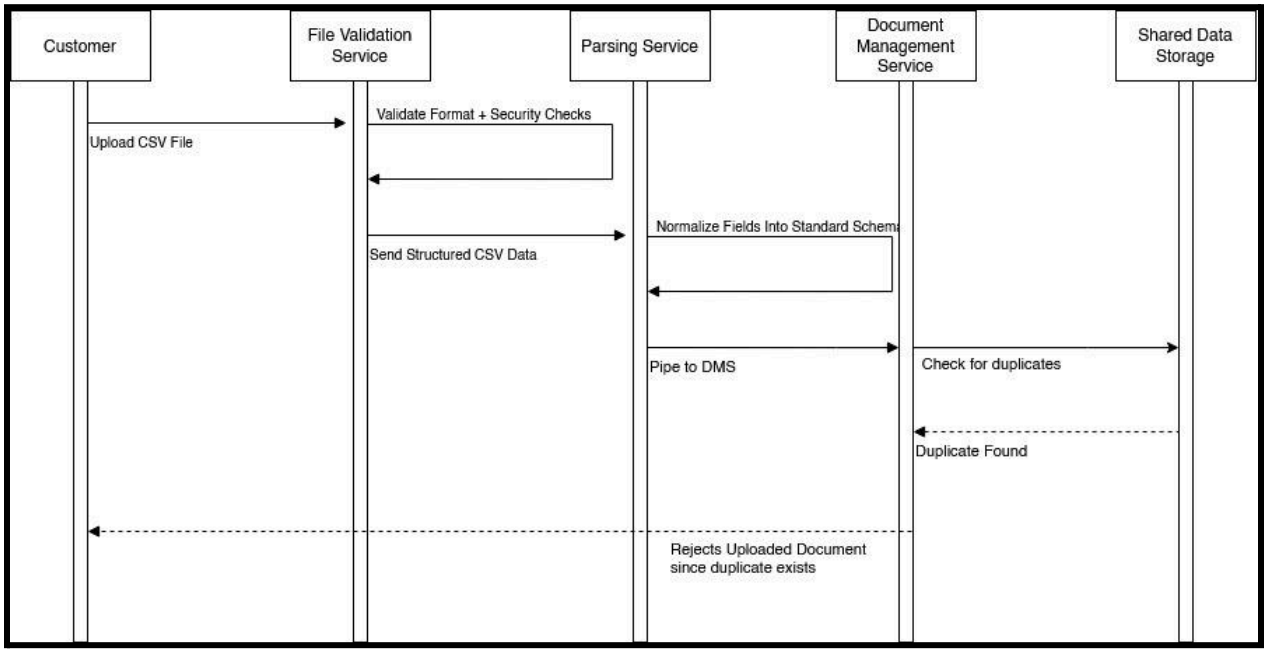
Scenario 3: Audio Submission Transcription and Processing

- A Customer uploads an audio file to the File Validation Service, which validates the format and runs security checks. The file is piped to the Audio Service for transcription, then forwarded to the DMS to check for duplicates. If none are found, the processed data and original file are stored and forwarded to a Data Validator for an Approve/Reject decision, after which it is moved to Accepted Data or removed.



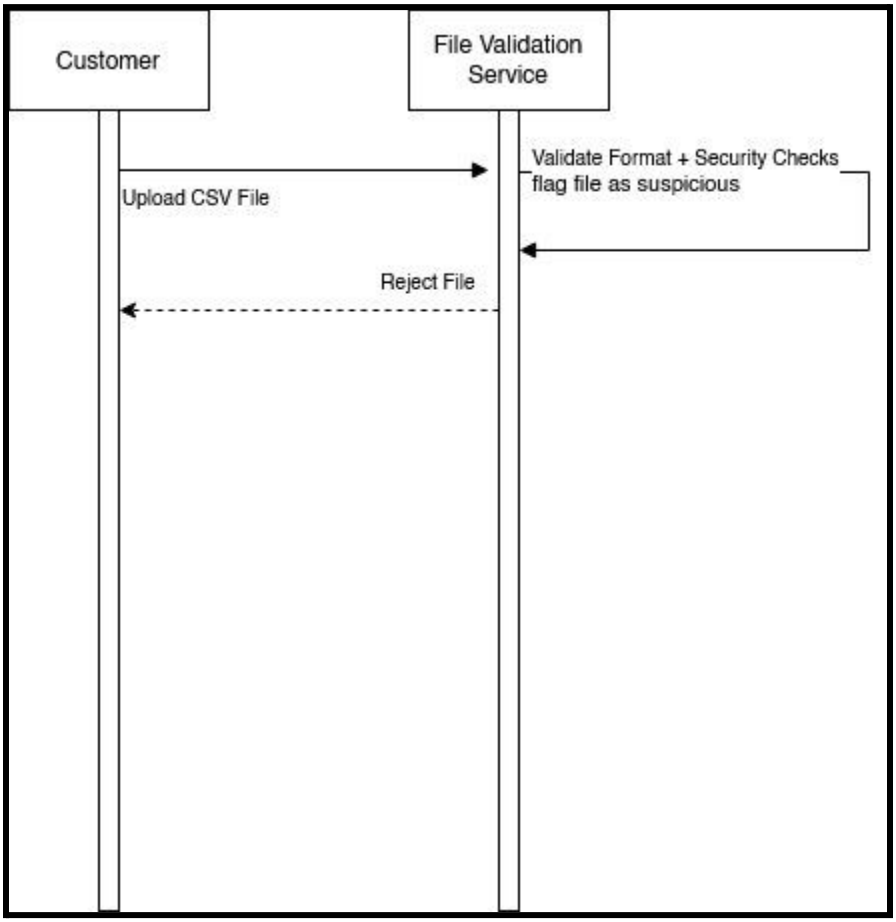
Scenario 4: Duplicate Submission Detection and Handling

- A Customer uploads a CSV file to the File Validation Service, which validates the format and runs security checks. The file is piped to the Parsing Service for normalization, then forwarded to the DMS, which queries Shared Data Storage and detects a duplicate. The DMS rejects the submission and notifies the Customer.



Scenario 5: Invalid or Malicious File Validation and Rejection

- A Customer attempts to upload a file to the File Validation Service, which validates the format and runs security checks. The file is flagged as suspicious and immediately rejected, preventing it from entering the system. The Customer is notified of the rejection.



Architecture Patterns Used

- Client-Server Architecture
  - Users and data validators interact only with the client interface, which communicates with backend services.
- Microservices Architecture
  - Independent services (OCR, parsing, validation, etc.) operate modularly and scale independently.
- Shared Data Architecture
  - Centralized data storage acts as the single source of truth for all services and clients.
- Layered Architecture
  - Clearly defined interaction boundaries:
    - Clients → APIs

        - Services → Data layer
        - Validators → Processed data only
- Pipe/Filter Architecture
    - Sequential data processing pipeline (e.g., OCR output flows directly into parsing and validation services).

Risks and Mitigation Strategies

Zero-Trust Data Handling
- Risk: Data cannot be implicitly trusted across microservices.
- Mitigation: Enforce validation at every stage and adopt zero-trust security principles.

System failure and Data Loss
- Risk: Failures across distributed components may cause data loss.
- Mitigation: Use database snapshots, rollback strategies, and service isolation.

OCR Inaccuracies
- Risk: OCR may produce incorrect outputs.
- Mitigation: Apply confidence scoring and human review workflows.

Inconsistent CSV Formats
- Risk: External agencies submit differently structured CSV files.
- Mitigation: Implement data cleaning and transformation services for normalization.

Audio Transcription Errors
- Risk: Transcription inaccuracies (automated or manual).
- Mitigation: Validation checks and review workflows.

High System Load
- Risk: Performance degradation under heavy batch uploads.
- Mitigation: Use scalable microservices, load balancing, and message queues.

Unauthorized Access
- Risk: Users accessing data beyond their permissions.
- Mitigation: Role-based access control (RBAC), least privilege, and multi-factor authentication (MFA).

System Downtime
- Risk: Service failures interrupt processing pipelines.

- Mitigation: Retry mechanisms, fallback to manual processing, and multi-region deployment (if budget allows).

Duplicate Claims

- Risk: Multiple submissions of the same claim via different channels.
- Mitigation: Use unique identifiers (e.g., SSN, application ID) and deduplication logic.

Latency

- Risk: Slow processing for large workloads.
- Mitigation: Use asynchronous processing with message queues and scalable cloud services.

Regulatory Compliance (HIPAA / GDPR)

- Risk: Non-compliance with legal data protection requirements.
- Mitigation: Data anonymization, strict access controls, and compliance-aware system design.